

[4. Obojsmerné asociácie](#)

[4.1 Obojsmerná 1:N - N:1 asociácia](#)

[4.2 Obojsmerná N:N asociácia](#)

[4.3 Stratégia dotazovania - Fetch type](#)

[Cvičenia](#)

4. Obojsmerné asociácie

4.1 Obojsmerná 1:N - N:1 asociácia

Pozri projekt [p41biN1](#)

Príklad - Knihy a ich vydavatelia: obojsmerná verzia asociácie z projektov [p33uniN1](#) aj [p34uni1N](#)

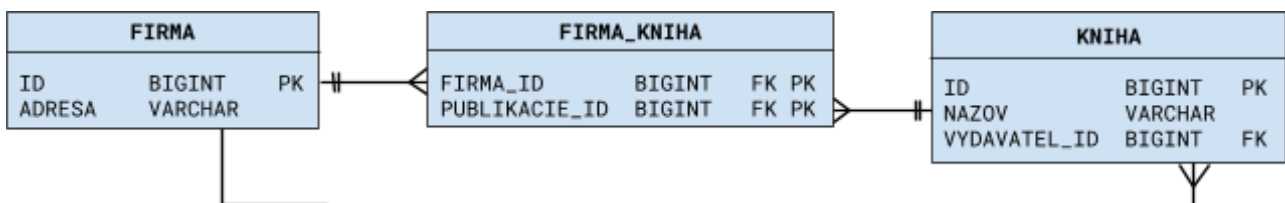


Entitné triedy - verzia 1 bez anotácií

```
@Entity
public class Kniha implements Serializable {
    @Id
    private Long id;
    private String nazov;
    private Firma vydavatel;
```

```
@Entity
public class Firma implements Serializable {
    @Id
    private Long id;
    private String adresa;
    private List<Kniha> publikacia;
```

Relačný diagram



JPA interpretuje referencie v triedach ako **dve nezávislé jednosmerné asociácie**. Obojsmerná asociácia je však tvorená dvoma jednosmernými, ktoré sú navzájom inverzné.

Entitné triedy - verzia 2

Informáciu, že jedna referencia je inverzná k druhej, musíme povedať pomocou anotácie `@OneToMany(mappedBy="vydavatel")`.

```
@Entity
public class Kniha implements Serializable {
    ...
    @ManyToOne
    private Firma vydavatel;
```

```
@Entity
public class Firma implements Serializable {
    ...
```

```
// @JoinColumn(name = "VYDAVATEL_FK")
@OneToMany(mappedBy="vydavatel")
private List<Kniha> publikacia;
```

Anotácia `@OneToMany(mappedBy = nahrádza @JoinColumn(name))` používané v prípade jednosmerných asociácií (pozri [3.4 Jednosmerná 1:N asociácia](#)). Pripomeňme, že pri `OneToMany` aj `ManyToOne` asociáciách je v databáze FK vždy na strane `Many`.

Dátový člen, ktorý je na **FK stĺpec** namapovaný, asociáciu vlastní, definuje a označuje sa **owner**. (V našom prípade je to `vydavatel` v triede `Kniha`.)

Zhrnutie: Parameter `mappedBy` prepája dátové členy na oboch stranách asociácie

- určuje, ktorý dátový člen referencovanej triedy je **owner** asociácie t.j. predstavuje inverznú asociáciu.
- používa sa len pri obojsmerných asociáciách a to na strane, ktorá nie je jej vlastníkom

Poznámka: Všimnite si, že `mappedBy` udáva meno atribútu triedy, na rozdiel od anotácie `@JoinColumn`, ktorá udáva meno FK stĺpca.

Aplikácia

Funkcia `create()` vytvorí objekty: firmu (vydavateľa) s niekoľkými knihami a vloží ich do databázy.

```
Firma f = new Firma();
f.setAdresa("Veda");
Kniha k1 = new Kniha();
k1.setNazov("1984");
Kniha k2 = new Kniha();
k2.setNazov("Animal Farm");

k1.setVydavatel(f);
k2.setVydavatel(f);
// pre vytvorenie FK v db nie je nutne
List<Kniha> kl = new ArrayList<>();
kl.add(k1);
kl.add(k2);
f.setPublikacie(kl);
```

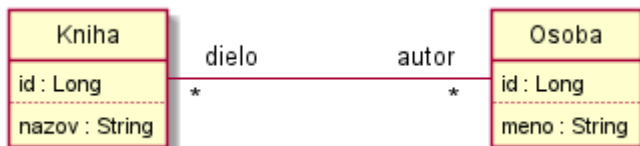
Poznámka: Pri vytváraní asociácie sme zadali referencie na oboch stranách asociácie. Pre správne vytvorenie hodnoty FK v databáze však stačí zadať len owner stranu t.j. `setVydavatel`. Pozri cvičenia.

Relačný diagram vygenerovaných tabuliek - štruktúra je rovnaký ako v prípade jednosmernej N:1 asociácie, pozri [3.3 Jednosmerná N:1 asociácia](#).

4.2 Obojsmerná N:N asociácia

Pozri projekt [p42biNN](#)

Príklad - Kniha a ich autori: Každá kniha pozná svojich autorov a každá osoba pozná svoje diela - knihy, ktorých je autorom.



Entitné triedy

Podobne ako v predchádzajúcom prípade, je potrebné povedať, že referencie `dielo` a `autor` sú navzájom inverzné. Keďže teraz sú obe strany anotácie `many`, použijeme anotáciu `@ManyToMany(mappedBy = )`, pričom **owner môže byť hociktorá strana**.

```
@Entity
public class Kniha implements Serializable {
    @Id
    private Long id;
```

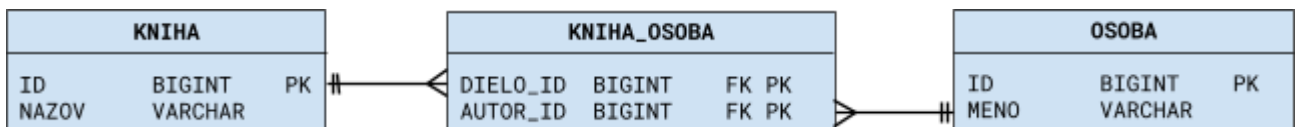
```
private String nazov;
@ManyToMany
private List<Osoba> autor;
```

```
@Entity
public class Osoba implements Serializable {
    @Id
    private Long id;
    private String meno;
    @ManyToMany(mappedBy="autor")
    private List<Kniha> dielo;
```

Aplikácia

Vytvorí a vloží do databázy niekoľko kníh s viacerými autormi, pričom niektoré knihy budú mať spoločných autorov. (Vid' UML object diagram v nasledujúcej časti)

Relačný diagram vygenerovaných tabuliek - Rovnako ako v prípade jednosmernej N:N asociácie, je vytvorená prepojujúca tabuľka



Poznámka: všimnite si názvy stĺpcov prepojujúcej tabuľky, sú odvodené od názvov dátových členov - zoznamov referencií.

4.3 Stratégia dotazovania - Fetch type

Pokračujeme s projektmi [p41biN1](#) a [p42biNN](#).

Aktivujeme v nich logovanie sql: do `persistence.xml` pridajte riadky,

```
<property name="eclipselink.logging.level.sql" value="FINE"/>
<property name="eclipselink.logging.parameters" value="true"/>
```

Aplikácie doplníme o funkcie na čítanie údajov z databázy a po ich spustení v logu pozorujeme SQL dopyty generované JPA.

Čítanie v prípade N:1 asociácie

Do projektu [p41biN1](#) doplníme funkciu, ktorá nájde knihu podľa kľúča a načíta všetky informácie o nej.

```
public static void readKniha(long id) {
    EntityManagerFactory emf = Persistence.createEntityManagerFactory("vsaPU");
    EntityManager em = emf.createEntityManager();
    Kniha k = em.find(Kniha.class, id);
    System.out.println("Nazov knihy: " + k.getNazov());
    System.out.println("Adresa vydavateľa: " + k.getVydavatel().getAdresa());
}
```

V logu si môžeme všimnúť, že metóda `find` načíta všetky údaje o hľadanej knihe naraz, vrátane údajov o jej vydavateľovi. (Naraz vyšle dva SELECT dotazy: pre dotazovanú entitu KNIHA aj referencovanú entitu FIRMA, výpisy názvu knihy a adresy vydavateľa nasledujú až za nimi.)

```
--SELECT ID, NAZOV, VYDAVATEL_ID FROM KNIHA WHERE (ID = ?)
    bind => [201]
--SELECT ID, ADRESA FROM FIRMA WHERE (ID = ?)
    bind => [101]
Nazov knihy: 1984
Adresa vydavateľa: Veda
```

Čítanie v prípade 1:N asociácie

Do toho istého projektu doplníme funkciu, ktorá nájde firmu podľa kľúča a načíta všetky informácie o nej.

```
public static void readFirma(long id) {
    EntityManagerFactory emf = Persistence.createEntityManagerFactory("vsaPU");
```

```

EntityManager em = emf.createEntityManager();
Firma f = em.find(Firma.class, id);
System.out.println("Adresa firmy: " + f.getAdresa());
System.out.println("Pocet publikacii: " + f.getPublikacie().size());

```

V logu si môžeme všimnúť, že metóda `find` vyšle teraz `SELECT` len pre dotazovanú entitu `Firma`. Dotaz na asociovanú entitu t.j. tabuľku `KNIHA` vyšle až neskôr volanie metódy `getPublikacie()`.

```

--SELECT ID, ADRESA FROM FIRMA WHERE (ID = ?)
      bind => [101]
Adresa firmy: Veda
--SELECT ID, NAZOV, VYDAVATEL_ID FROM KNIHA WHERE (VYDAVATEL_ID = ?)
      bind => [101]
Pocet publikacii: 2

```

Zhrnutie:

- V prvom prípade, keď išlo o asociáciu N:1, sa s dotazovaným objektom načítali údaje aj s asociovaným objektom. Táto stratégia - **FetchType** sa nazýva **EAGER**.
- V druhom prípade, keď išlo o asociáciu 1:N, sa údaje o asociovaných objektoch nenačítali hneď, ale až keď boli potrebné. Táto stratégia - **FetchType** sa nazýva **LAZY**.

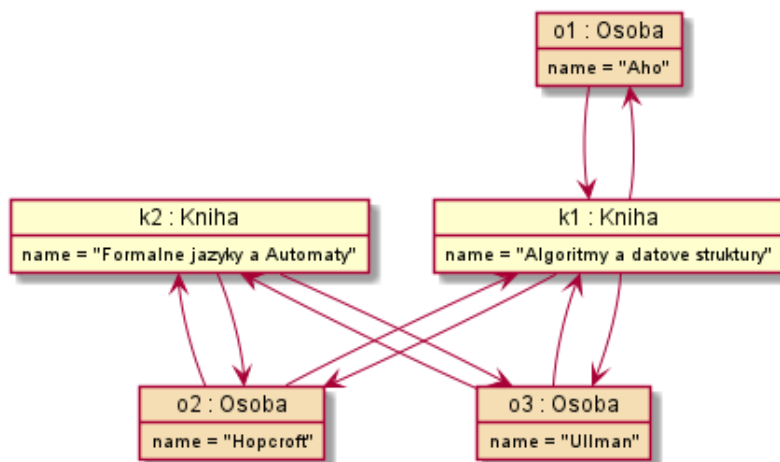
Uvedené správanie môžeme zmeniť v anotáciách parametrom `fetch`, napríklad:

`@OneToMany(fetch=FetchType.EAGER)` alebo `@ManyToOne(fetch=FetchType.LAZY)`

Použiť sa dá aj pri `@ManyToMany`, `@OneToOne`.

Čítanie v prípade N:N asociácie

Nastavenie **EAGER** v prípade viacnásobných referencií môže spôsobiť zbytočné záťažové problémy. Knihy, osoby a ich vzájomné referencie vytvorené aplikáciou v projekte [p42biNN](#) zobrazuje UML diagram objektov:



Pre zoznamy referencií na oboch stranách asociácie nastavíme `FetchType.EAGER`.

```

@Entity
public class Kniha implements Serializable {
    ...
    @ManyToMany(fetch = FetchType.EAGER)
    private List<Osoba> autor;
}

```

```

@Entity
public class Osoba implements Serializable {
    ...
    @ManyToMany(mappedBy="autor", fetch = FetchType.EAGER)
    private List<Kniha> dielo;
}

```

Keď teraz v aplikácii vyhladáme podľa kľúča jedinou osobu, uvidíme v logu, že sa tým spustila celá lavína SQL dopytov, ktorými sa rekurzívne čítajú aj ďalšie knihy a osoby:

```

SELECT ID, MENO FROM OSOBA WHERE (ID = ?)
      bind => [333]
SELECT t1.ID, t1.NAZOV FROM KNIHA_OSOBA t0, KNIHA t1 WHERE ((t0.autor_ID = ?) AND (t1.ID = t0.dielo_ID))

```

```

        bind => [333]
SELECT t1.ID, t1.MENO FROM KNIHA_OSOBA t0, OSOBA t1 WHERE ((t0.dielo_ID = ?) AND (t1.ID = t0.autor_ID))
        bind => [52]
SELECT t1.ID, t1.NAZOV FROM KNIHA_OSOBA t0, KNIHA t1 WHERE ((t0.autor_ID = ?) AND (t1.ID = t0.dielo_ID))
        bind => [53]
SELECT t1.ID, t1.MENO FROM KNIHA_OSOBA t0, OSOBA t1 WHERE ((t0.dielo_ID = ?) AND (t1.ID = t0.autor_ID))
        bind => [51]
SELECT t1.ID, t1.NAZOV FROM KNIHA_OSOBA t0, KNIHA t1 WHERE ((t0.autor_ID = ?) AND (t1.ID = t0.dielo_ID))
        bind => [54]

```

Načítali sa všetky objekty, ku ktorým vedie od dotazovanej osoby cesta v grafe referencií.

Cvičenia

Cvičenie 1 - modifikácie [p41biN1](#)

Vo funkcii `create` sme zadali referencie na oboch stranách asociácie: Knihám sme zadali referenciu na firmu, ktorá ich vydáva a firme sme vložili vydávané knihy do zoznamu diel.

a) Upravte funkciu `create` tak, že iba knihám zadáte referenciu na vydavateľa. Po jej zavolaní sa presvedčte, že v databáze je FK stĺpec správne vyplnený.

Následne použite funkciu `readFirma` na načítanie údajov z databázy a presvedčte sa, že v zozname diel sú už referencie na vydávané knihy.

b) Upravte funkciu `create` tak, že teraz len firme vložíte vydávané knihy do zoznamu diel (knihám však vydavateľa nedostavíte). Presvedčte sa, či po zavolaní funkcie bude v databáze FK stĺpec správne vyplnený.

Cvičenie 2 - modifikácie [p42biNN](#)

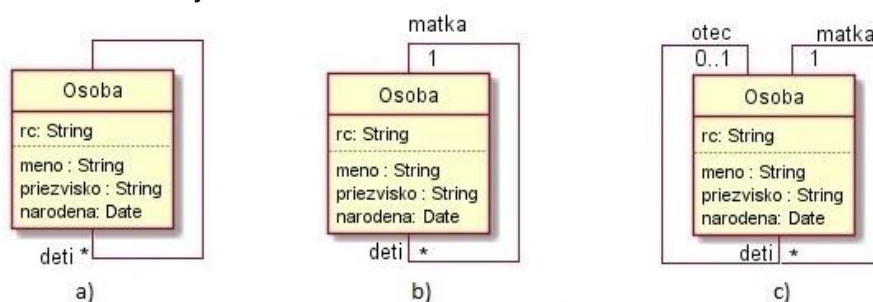
Upravte funkciu `create` tak, že vytvoríte dve knihy a ich autorov: Brave new world, ktorej autorom je Huxley a 1984 od Orwella. Osobám priradíte ich diela správne ale zadávaní autorov kníh ich vymeníte (ako autora Brave new world uvediete Orwella autorom 1984 bude Huxley).

a) Po zápise do databázy sa v nej (v prepojujacej tabuľke) presvedčte či sú diela priradené autorom správne alebo naopak.

b) skúste zmeniť anotácie `ManyToMany` tak aby dielo bolo owner asociácie. Vytvorte databázu znovu a presvedčte sa, že teraz sú v nej diela priradené správne.

Vysvetlenie: Pri obojstrannej asociácii je pre zápis do databázy určujúca referencia na strane owner.

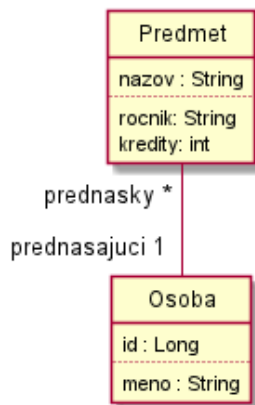
Cvičenie 3. Skúste reimplementovať cvičenie 6 z časti JPA3, tak, že asociácie medzi deťmi a rodičmi budú obojsmerné:



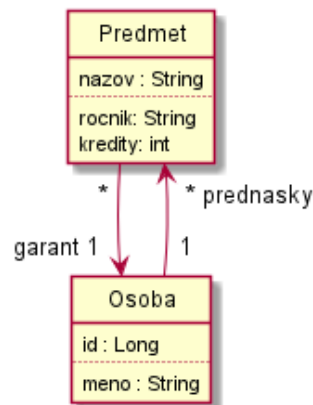
Poznámky: Vo verzii a) diagramu nie je zadáný názov pre opačnú stranu asociácie. Názov dátového člena pre opačnú stranu asociácie si môžete vhodne zvoliť sami.

Pokiaľ budete mať problém, nájsť uspokojivé riešenie pre verziu c) UML diagramu, zvážte úpravu modelu tak, aby vyjadrovala skutočnosť, že otcom musí byť muž a matkou žena. Pozri [cvičenie 6 v časti JPA-6](#).

Cvičenie 4. Ilustruje rozdiel medzi obojsmernou asociáciou a dvoma jednosmernými asociáciami.



UML- A obojsmerná asociácia



UML-B dve jednosmerné asociácie

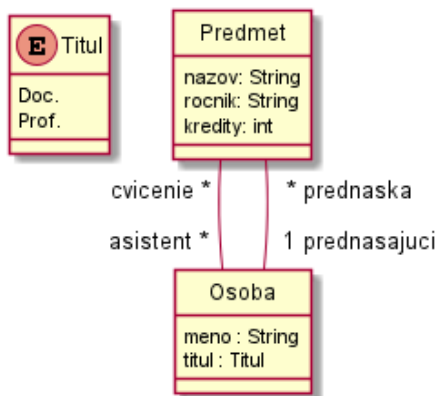
Pre oba UML diagramy si najprv nakreslite, ako bude vyzerat štruktúra zodpovedajúcej databázy.

Potom pre každý diagram vytvorte projekt, implementujte v ňom entitné triedy podľa diagramu a jednoduchý testovací program, ktorý vytvorí nejaké predmety, osoby a asociácie medzi nimi.

V prípade B si overte, že garant predmetu nemusí predmet prednášať, t.j. nemusí ho mať medzi svojimi prednáškami. Prednášajúci môže byť iná osoba.

Po spustení programov si overte, či vygenerované databázy má takú štruktúru ako ste očakávali.

Cvičenie 5. Pre UML diagram tried znázornených na obrázku navrhnete štruktúru databázy a nakreslite si jej relačný diagram. Poznámka symbol  označuje enumeračný typ.



Vytvorte nový projekt a v ňom implementujte:

A) entitné triedy, tak aby odpovedali vášmu návrhu DB (kľúčové atribúty môžu byť autogenerované) pričom asociácie sú obojsmerné a majú kardinalitu tak ako je to na diagrame. Implementujte funkciu create, ktorá vytvorí dva predmety a troch osoby, pričom

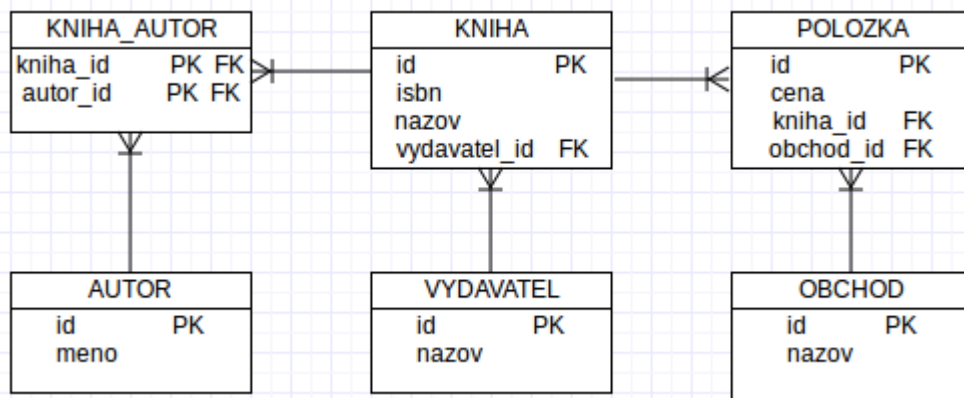
1. osoba bude prednášať 1. predmet,
2. osoba bude prednášať aj cvičiť 2. predmet
3. osoba bude cvičiť 1. aj 2. predmet

Jej spustením vygenerujete databázu. Overte, či má štruktúru akú ste navrhli.

B) funkciu `void read(String meno)`, ktorá dostane meno osoby, nájde všetky predmety, ktoré cvičí (ako asistent). Na obrazovku vypíše jeho názov a meno prednášajúceho.

C) funkciu `void release(Osoba o)`, ktorá dostane ako parameter osobu, zistí či taká osoba je v DB, ak áno odstráni ju, pričom odstráni aj všetky odkazy na ňu, t.j. nesmie ostať už ani prednášajúcim ani cvičiacim/asistentom žiadneho predmetu.

Cvičenie 6. Na obrázku je ER diagram zobrazujúci štruktúru databázy portálu o knihách.



- Navrhnete entitné triedy odpovedajúce ER diagramu, (t.j. JPA by z entitných tried mal automaticky vygenerovať tabuľky tak, ako sú zobrazené na diagrame):
 - Načrtnite si najprv UML diagram tried
 - Pri návrhu asociácií zoberte do úvahy, akým spôsobom budete chcieť navigovať medzi objektami entitných tried.
- Implementujte funkciu, ktorá vyhľadá knihu podľa isbn.
- Implementujte funkciu, ktorá vyhľadá autora podľa mena.
- Implementujte funkciu, ktorá načíta z databázy všetky obchody a pre každý obchod vypíše všetky údaje o knihách (aj autoroch) v ponuke.
- Implementujte funkciu, ktorá dostane ako argument objekt triedy Autor a vráti zoznam všetkých obchodov (objektov triedy Obchod), ktoré ponúkajú knihu od daného autora (t.j. je jedným z jej autorov). Funkcia môže predpokladať, že objekt, ktorý dostane ako argument je manažovaný, t.j. bol načítaný z databázy.
- Implementujte program, ktorý otestuje funkčnosť oboch funkcií. Pre naplnenie databázy testovacími údajmi implementujte funkciu, ktorá importuje do DB dáta zo súboru [data4cv45.csv](#).

Cvičenie 7. Modifikujte návrh štruktúry databázy z **cvičenia 6**, tak aby zaručoval, že cena knihy bude vo všetkých obchodoch rovnaká. Prispôbte tomu entitné triedy, reimplementujte funkcie a program